KP26 The Class Imbalance Problem

> Level: undergraduate, year 3
>
> Pattern Recognition | Chapter 4, §4.23

Slides: read and download

Learning objectives

- Explain why class imbalance harms classifiers and why accuracy can deceive.
- Name the data-level and the cost-sensitive cures and compare their trade-offs.
- Compute recall, precision, accuracy and balanced accuracy from the rare points caught (TP) and the false alarms (FP), and check the numbers in a table.

KP26 The Class Imbalance Problem

One-sentence summary: When one class has ten thousand samples and another only ten, a classifier that "seeks simplicity" naturally ignores the rare class, which is often the very class we care about.

1. How the experiment works

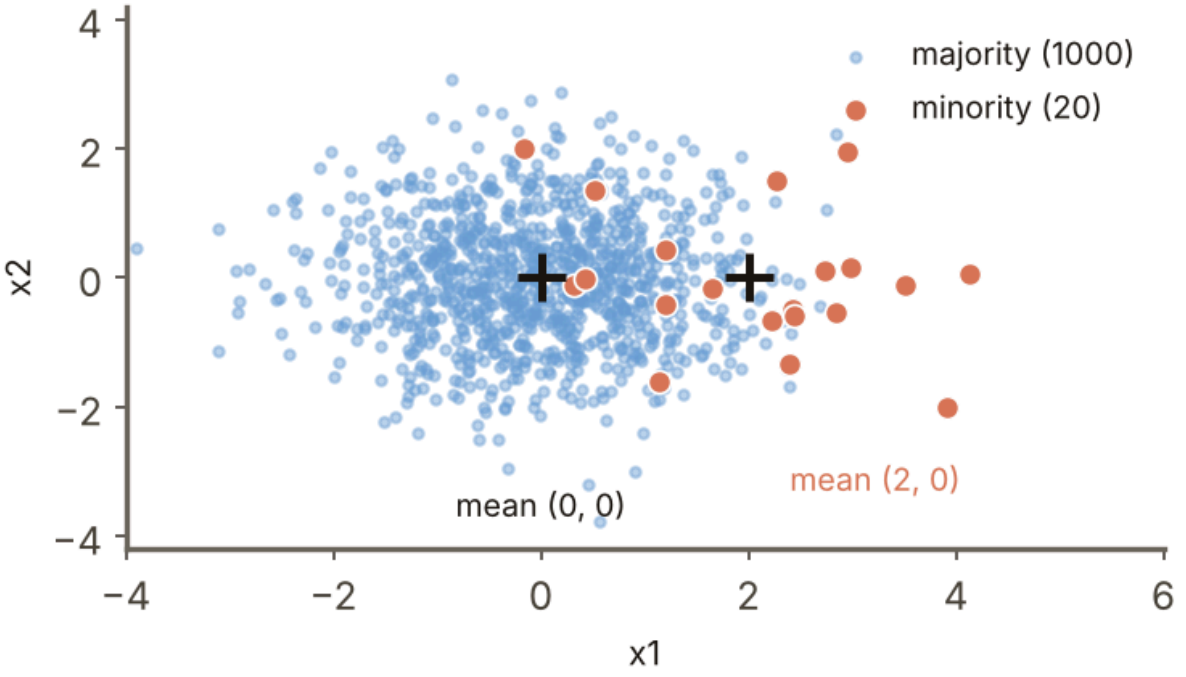
Plain explanation: The AdaBoost of KP25 degrades with noisy labels. Is a high accuracy reliable when one class has only 20 samples? First, how the experiment is made.

- **Class imbalance**: one class has very many samples and another very few; common in text classification, rare diseases, oil-spill detection.

Data (designed here, program-generated): majority $N((0,0), I)$, rare $N((2,0), I)$, independent coordinates, standard deviation 1; training 1000 + 20 points, one fixed test set of 5000 + 100.

Model and repeats: RBF-kernel SVM ($e^{-0.5\|x-z\|^2}$, $C = 1$); one training set per random seed 0 to 19, 20 runs, averaged.

Figure `lec05-kp26-setup`: Scatter of one training set: 1000 blue majority points centred at (0,0) and 20 orange rare points centred at (2,0), overlapping; below it the training, test, repeat count and RBF SVM parameters.



train: 1000 ~ N((0,0), I) and 20 ~ N((2,0), I)

test (one fixed set): 5000 + 100 from the same two

repeat with training seeds 0 to 19, average the counts

RBF SVM: exp(-0.5 |x-z|^2), C = 1

Experiment details (reproducible): implemented with scikit-learn in Python (SVC, DecisionTreeClassifier, AdaBoostClassifier, version 1.9.1 defaults except as listed). On pages 1 to 3 and 5 to 7 the test set is fixed at 5000 + 100 (random seed 998) and the training sets use seeds 0 to 19; the sampling of oversampling and undersampling reuses the training seed. On page 4 the test set is fixed at 2000 + 2000 (seed 997) and the training sets again use seeds 0 to 19, 20 runs, with the class centres differing only along the x_1 axis by 4 or 1.5. The tree is CART (Gini impurity, random seed 0); AdaBoost uses depth-1 decision stumps, 100 rounds and learning rate 1. The SVM class weights are $n/(2n_c)$: rare $1020/(2 \times 20) = 25.5$, majority $1020/(2 \times 1000) = 0.51$, a ratio of 50. A mean count is the mean of 20 integer counts, so it can

have decimals (for example 5.85 = 117 / 20). TP in each cell of page 4 (of 2000 rare test points): easy task 1950.9, 1831.8, 1335.1, 1561.1, 863.8; hard task 1571.65, 298, 0.35, 0.65, 0.

2. From counts to measures

Plain explanation: Every number comes from this experiment: 20 rare points among 1000 majority ones, tested on 5100 new points where the rare class is only 2%. How is a classifier scored?

- **Recall**: share of the 100 rare points caught, $= \mathrm{TP}/100$; **precision**: share of points called rare that are, $= \mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$. TP: rare points caught; FP: false alarms.

Accuracy $= (\mathrm{TP}+\mathrm{TN})/5100$, $\mathrm{TN} = 5000 - \mathrm{FP}$. **Always the majority**: TP 0, FP 0, recall $0/100 = 0$, accuracy $5000/5100 = 98.0\%$.

Example (SVM, mean of 20 runs): TP 5.85, FP 2; recall $5.85/100 = 5.9\%$, precision $5.85/(5.85+2) = 74.5\%$, accuracy $(5.85+4998)/5100 = 98.1\%$.

Figure `lec05-kp26-metrics`: From counts to measures: always the majority has TP 0 and FP 0, recall 0 and accuracy 98.0%; the SVM mean of 20 runs has TP 5.85 and FP 2, recall 5.9%, precision 74.5% and accuracy 98.1%, with the substituted formulas.

test: 5000 majority + 100 rare; TP = rare caught, FP = false alarms

recall = TP/100 precision = TP/(TP+FP)

accuracy = (TP + 5000 - FP)/5100

always say majority: TP 0, FP 0

recall 0/100 = 0%

accuracy (0 + 5000)/5100 = 98.0%

standard SVM (mean of 20 runs): TP 5.85, FP 2

recall 5.85/100 = 5.9%

precision 5.85/(5.85+2) = 74.5%

accuracy (5.85 + 4998)/5100 = 98.1%

98.0% vs 98.1%: the 100 rare points are only 2% of the test, so accuracy can hardly see them

3. The simplicity preference

Plain explanation: The SVM catches only 5.85 rare points on average and treats the other 94 as majority. Why does a classifier ignore the rare class?

Reasoning (source): for generalization we ask for simple classifiers (regularization, pruning); a simple hypothesis ignores rare cases, and ignoring 20 points barely changes the total cost.

SVM: C penalizes a misclassified point (KP15). $C = 0.01, 0.1$: TP 0, FP 0; $C = 1$: TP 5.85; $C = 100$: TP 16.85, FP 10.9. Recall $= \mathrm{TP}/100$; accuracy about 98%.

Tree (CART, at least k points per leaf): $k = 1$: TP 23.25, FP 73.1; $k = 5$: 21, 29.85; $k = 20$: 9.45, 12.55; $k = 50$: 0, 0.

Figure `lec05-kp26-simple`: SVM TP is 0, 0, 5.85, 13.45 and 16.85 for C of 0.01, 0.1, 1, 10 and 100; tree TP is 23.25, 21, 9.45 and 0 for minimum leaf sizes 1, 5, 20 and 50; the table also lists FP, recall and accuracy.

SVM, larger C = less regularization

C	TP	FP	recall	accuracy
0.01	0	0	0.0%	98.0%
0.1	0	0	0.0%	98.0%
1	5.85	2	5.9%	98.1%
10	13.45	7.35	13.5%	98.2%
100	16.85	10.9	16.9%	98.2%

decision tree, larger leaf = simpler

min leaf	TP	FP	recall	accuracy
1	23.25	73.1	23.3%	97.1%
5	21	29.85	21.0%	97.9%
20	9.45	12.55	9.5%	98.0%
50	0	0	0.0%	98.0%

means of 20 runs; recall = TP/100; accuracy stays near 98%

4. Difficulty and data size

Plain explanation: The simpler the classifier, the less it cares about 20 points. Is it always bad whenever the classes are imbalanced?

Source (Japkowicz): a large imbalance need not hurt; read it with task difficulty and training size. An easy task or much data may be fine; a hard task or too few points makes even a small imbalance fatal.

How (designed here): RBF SVM ($C = 1, \gamma = 0.5$); training majority:rare $= 100{:}100$, $1000{:}100$, $1000{:}10$, $10000{:}100$, $10000{:}10$; fixed test 2000 + 2000, recall $= \mathrm{TP}/2000$; centres 4 (easy) or 1.5 (hard) apart.

Results: easy-task recall 97.5, 91.6, 66.8, 78.1, 43.2%; hard task 78.6, 14.9, 0, 0, 0%. With 10 rare points even the easy task does poorly.

Figure `lec05-kp26-grid`: Rare-class recall (TP over 2000): easy task 97.5% at 100:100 and 66.8% at 1000:10; hard task 14.9% at 1000:100 and about 0% for the other

imbalanced settings; each cell shows its TP.

rare-class recall = TP/2000 on a fixed test set of 2000 + 2000

majority : rare	easy (means 4 apart)	hard (1.5 apart)
100 : 100	**97.5%** TP 1950.9 / 2000	**78.6%** TP 1571.65 / 2000
1000 : 100	**91.6%** TP 1831.8 / 2000	**14.9%** TP 298 / 2000
1000 : 10	**66.8%** TP 1335.1 / 2000	**0.0%** TP 0.35 / 2000
10000 : 100	**78.1%** TP 1561.1 / 2000	**0.0%** TP 0.65 / 2000
10000 : 10	**43.2%** TP 863.8 / 2000	**0.0%** TP 0 / 2000

RBF SVM, C = 1, gamma = 0.5; means of 20 runs

the count of rare points matters as much as the ratio

5. Changing the data

Plain explanation: Imbalance is not always bad, but on a hard task 1000 to 100 presses the recall to 14.9%. The first cure changes the data: copy the rare class or drop majority points, with what result?

Method: oversampling draws 980 more from the 20 rare points with replacement, so training becomes 1000 + 1000; undersampling keeps 20 of the 1000 majority points at random without replacement, so training becomes 20 + 20. The same SVM ($C = 1$).

Results (TP / FP): plain 5.85 / 2; oversampling 68.15 / 605.55; undersampling 77.05 / 756.7. Example: oversampling recall $68.15/100 = 68.2\%$, precision $68.15/(68.15 + 605.55) = 10.1\%$.

Price: the rare class is found, but precision drops to about 10% and accuracy to 85% to 88%.

Figure `lec05-kp26-data`: Data-level cures: plain SVM TP 5.85, FP 2; oversampling 68.15, 605.55; undersampling 77.05, 756.7; the table also lists training size, recall, precision and accuracy.

SVM (RBF, C = 1); test 5000 + 100; means of 20 runs

	TP	FP	recall	precision	acc.
plain train 1000 + 20	**5.85**	**2**	**5.9%**	**74.5%**	**98.1%**
oversample train 1000 + 1000	**68.15**	**605.55**	**68.2%**	**10.1%**	**87.5%**
undersample train 20 + 20	**77.05**	**756.7**	**77.1%**	**9.2%**	**84.7%**

oversample: copy the 20 rare points, with replacement, up to 1000

undersample: keep 20 of the 1000 majority points at random

FP 2 → over 600: caught more, but far more false alarms

6. Focused resampling

Plain explanation: Oversampling and undersampling were random, with many false alarms. Would it be better to drop only the majority points far from the boundary, which carry no information?

Method: delete points far from the boundary (source). Steps: a first SVM ($C = 1$) scores the 1000 majority points; keep the m with the largest scores (most like the rare class), retrain on 20 rare + m; $m = 20, 60, 100, 200$.

Results (TP / FP / accuracy): 20 kept: 82.7 / 2126.05 / 58.0%; 60: 11.45 / 55.5 / 97.2%; 100: 8.3 / 32.55 / 97.6%; 200: 5.95 / 2 / 98.1%; all 1000 = plain SVM.

Meaning: the right target distribution is open (source): 20 vs 60 kept differ hugely; 2126 false alarms at 20 kept, 757 for random undersampling.

Figure `lec05-kp26-focus`: Focused undersampling keeping 20, 60, 100 and 200 majority points: TP 82.7, 11.45, 8.3 and 5.95, FP 2126.05, 55.5, 32.55 and 2, with recall and accuracy; the last row is the plain SVM keeping all.

first SVM scores the majority; keep the highest scores

kept	train	TP	FP	recall	accuracy
20	**20+20**	**82.7**	**2126.05**	**82.7%**	**58.0%**
60	**60+20**	**11.45**	**55.5**	**11.5%**	**97.2%**
100	**100+20**	**8.3**	**32.55**	**8.3%**	**97.6%**
200	**200+20**	**5.95**	**2**	**6.0%**	**98.1%**
1000	1000+20	5.85	2	5.9%	98.1%

last row: keep all 1000 = the plain SVM

kept 20: TP 83 but FP 2126 and accuracy 58%

kept 60: back down to TP 11. How many to keep?

7. Changing the cost

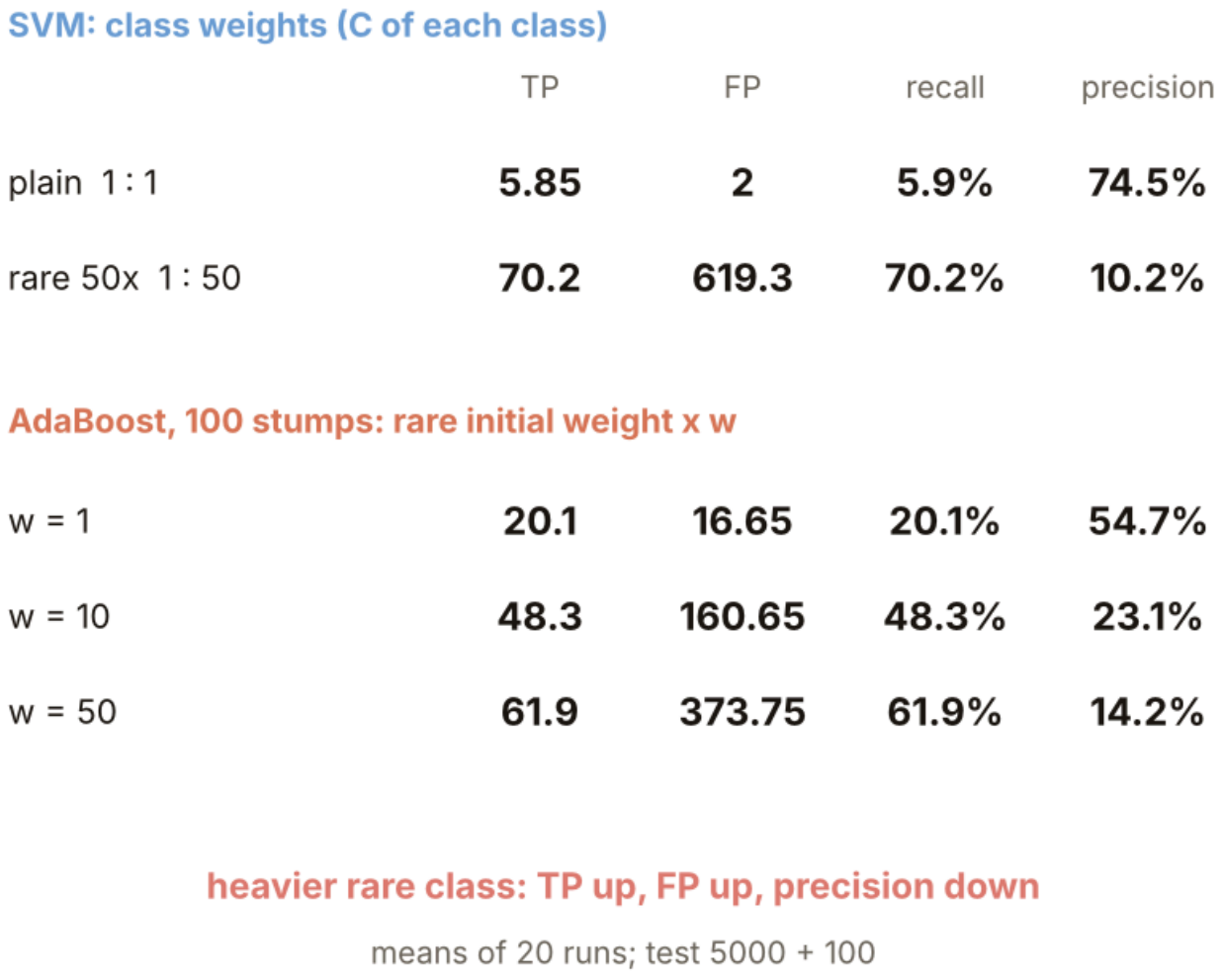
Plain explanation: Focused undersampling: how many to keep is hard, and 20 kept gave 2126 false alarms. What does the cost-sensitive cure, changing the classifier and not the data, achieve?

Method (source): weight the rare class more. SVM: rare-class C 50 times larger (weights 25.5 vs 0.51); AdaBoost: 100 stumps, rare initial weight $\times w$, $w = 1, 10, 50$, then normalized.

SVM: TP 70.2, FP 619.3; recall 70.2%, precision $70.2/(70.2 + 619.3) = 10.2\%$.

AdaBoost (TP / FP): $w = 1$: 20.1 / 16.65; $w = 10$: 48.3 / 160.65; $w = 50$: 61.9 / 373.75. Recall 20.1%, 48.3%, 61.9%; precision 54.7%, 23.1%, 14.2%.

Figure `lec05-kp26-cost`: Cost-sensitive: the SVM with a 50 times rare-class C has TP 70.2 and FP 619.3; AdaBoost with w of 1, 10 and 50 has TP 20.1, 48.3 and 61.9 and FP 16.65, 160.65 and 373.75.



8. To remember

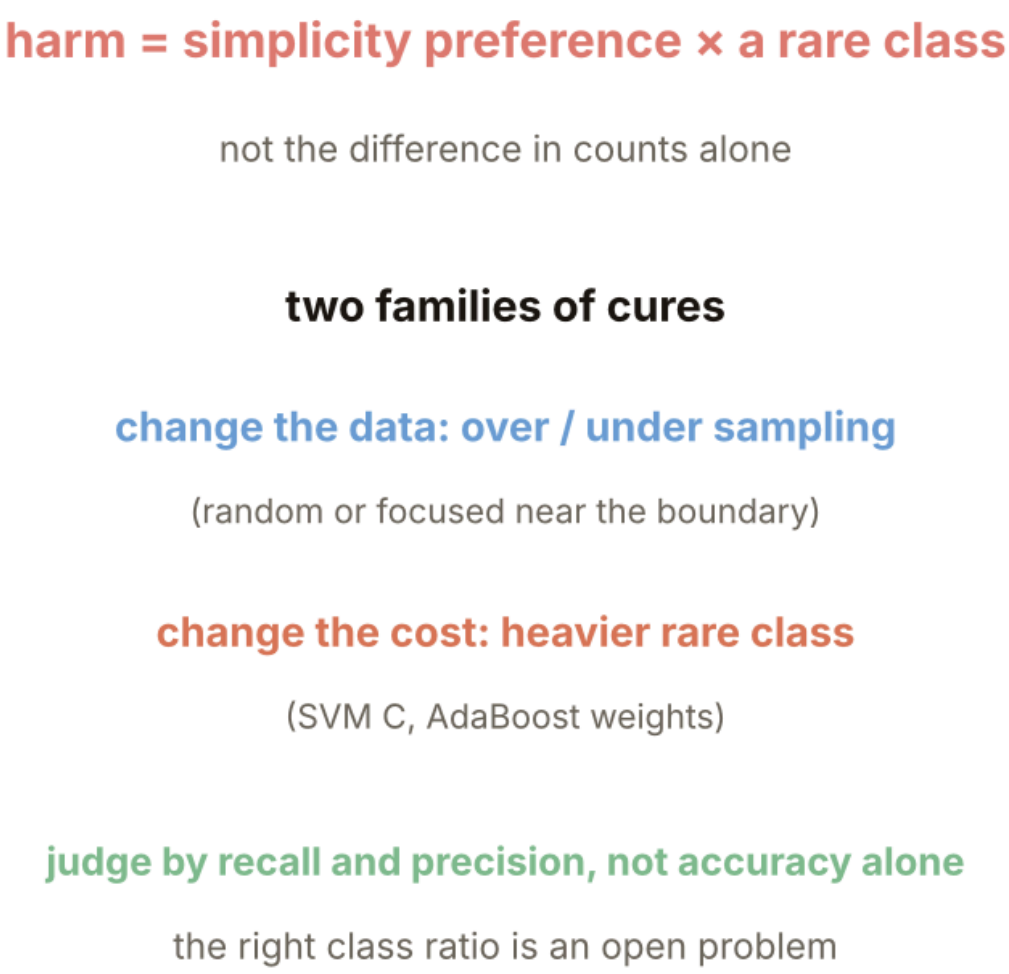
Plain explanation: Changing the cost, like changing the data, raised TP and FP together and cut precision. What should be remembered, and how should one evaluate?

To remember (source): the harm does not come from the numbers being different but from the interaction of the simplicity preference with the rare class. Two cures: change the data (resampling) or change the cost (weighting).

Common misconception: a high accuracy is enough. Evaluate by looking at recall and precision, not only accuracy; always saying the majority also scores 98.0%.

Reminder (source): class imbalance matters a great deal in practice; a designer must be aware of it and alert to the cures.

Figure `lec05-kp26-recap`: The key points of class imbalance: the harm comes from the simplicity preference times the rare class; the two cures are changing the data or the cost; look at recall and precision.



Extension: comparing the cures, balanced accuracy and practice

comparing the cures and balanced accuracy

Plain explanation: What to remember is the trade-off. With the four treatments in one table and a measure that sees the rare class, does any clearly win?

Formal definition: balanced accuracy $= (\text{recall} + \text{specificity})/2$, specificity $= \text{TN}/5000$; always the majority $(0 + 1)/2 = 0.50$. Example: plain SVM $(0.0585 + 4998/5000)/2 = 0.53$.

SVM (TP / FP / balanced accuracy): plain 5.85 / 2 / 0.53; oversampling 68.15 / 605.55 / 0.78; undersampling 77.05 / 756.7 / 0.81; cost-sensitive 70.2 / 619.3 / 0.79.

Tree (min leaf 5): plain 21 / 29.85 / 0.60; oversampling 33.75 / 101.45 / 0.66; undersampling 77.85 / 912.6 / 0.80; cost-sensitive 43.15 / 227.7 / 0.69. None clearly wins.

Figure `lec05-kp26-table`: Table of TP, FP, recall, precision, accuracy and balanced accuracy for four treatments of an SVM and a decision tree: SVM balanced accuracy 0.53, 0.78, 0.81 and 0.79.

	TP	FP	recall	prec.	acc.	bal.
SVM						
plain	**5.85**	**2**	**6%**	**75%**	**98%**	**0.53**
oversample	**68.15**	**605.55**	**68%**	**10%**	**88%**	**0.78**
undersample	**77.05**	**756.7**	**77%**	**9%**	**85%**	**0.81**
cost-sensitive	**70.2**	**619.3**	**70%**	**10%**	**87%**	**0.79**
decision tree (min leaf 5)						
plain	**21**	**29.85**	**21%**	**41%**	**98%**	**0.60**
oversample	**33.75**	**101.45**	**34%**	**25%**	**97%**	**0.66**
undersample	**77.85**	**912.6**	**78%**	**8%**	**82%**	**0.80**
cost-sensitive	**43.15**	**227.7**	**43%**	**16%**	**94%**	**0.69**

bal. = (recall + TN/5000)/2; always majority: bal. 0.50

practice

Plain explanation: Balanced accuracy leaves always saying the majority at 0.50. Back to the definitions: compute by hand the measures of A, which says majority for

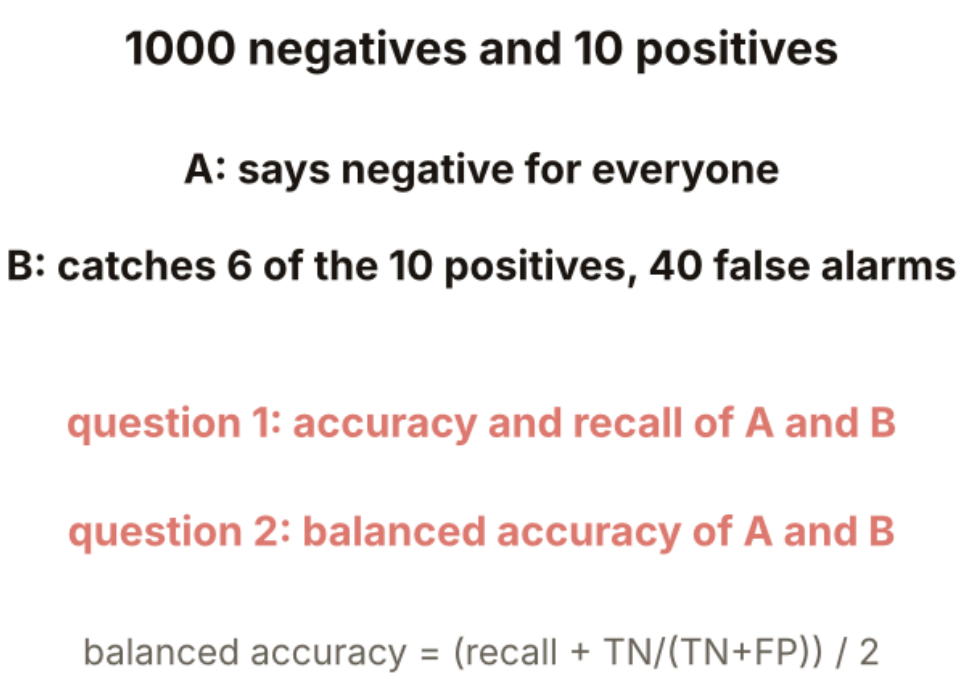
everyone, and of B, which catches 6: what are they?

Practice (setup): 1000 negatives and 10 positives (the rare class). A says negative for everyone; B catches 6 of the 10 positives and raises 40 false alarms.

Question 1: what are the accuracy and recall of A and B?

Question 2: what are the balanced accuracies of A and B?

Figure `lec05-kp26-practice`: Practice figure: 1000 negatives and 10 positives; A says negative for everyone and B catches 6 with 40 false alarms.



solution

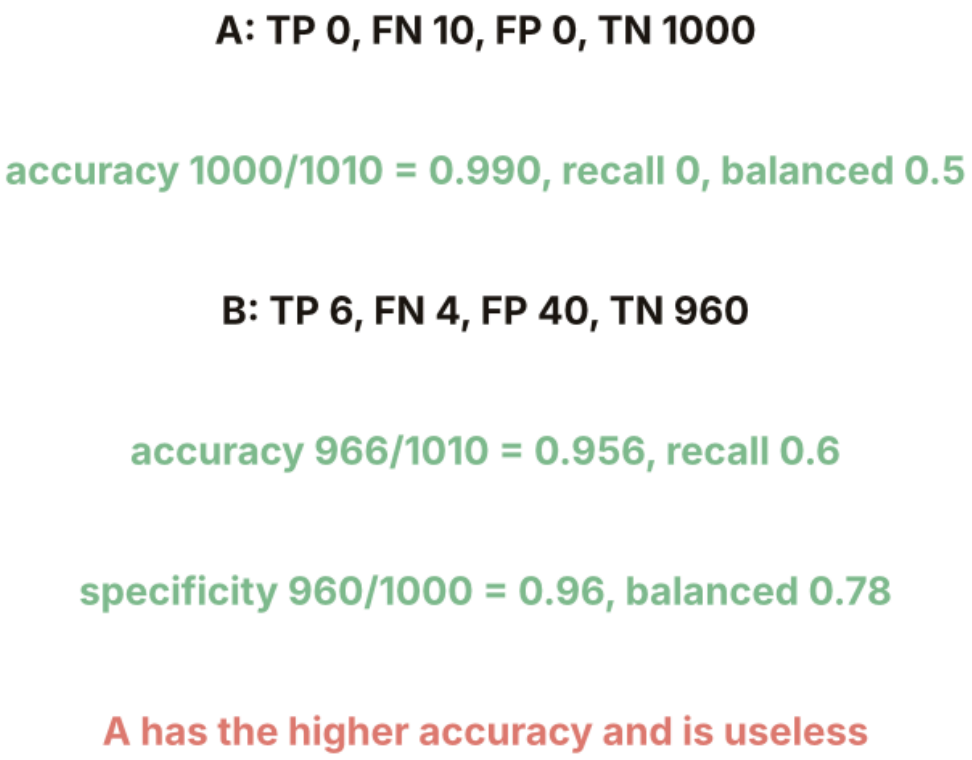
Plain explanation: The practice asked for the accuracy, recall and balanced accuracy of A and B. Worked out in full, what are the answers?

Question 1: A: TP 0, FN 10, FP 0, TN 1000; accuracy $1000/1010 = 0.990$, recall 0. B: TP 6, FN 4, FP 40, TN 960; accuracy $966/1010 = 0.956$, recall 0.6.

Question 2: A: $(0 + 1)/2 = 0.5$. B: specificity $960/1000 = 0.96$, $(0.6 + 0.96)/2 = 0.78$.

Check: A has the higher accuracy yet catches no rare point; B has a precision of only $6/46 = 0.13$, which is the trade-off.

Figure `lec05-kp26-practice-sol`: A: accuracy 0.990, recall 0 and balanced accuracy 0.5; B: accuracy 0.956, recall 0.6 and balanced accuracy 0.78.



Terms and source

中文	English
類別不平衡	Class imbalance
召回率	Recall

Source: Chapter 4, KP26, §4.23 (read 2026-10-25). That class imbalance harms classifiers, that the harm comes from the interaction of the simplicity preference with the rare class, that it must be read with task difficulty and data size, the data-level and cost-sensitive cures, focused resampling, the open problem of the target distribution and the 'to remember' line follow the source; the experiment setup, the count formulas of the measures, the sweeps of C and leaf size, the difficulty and data-size table, the comparison of the cures, balanced accuracy and the practice were designed and recomputed for this lecture ([added], (program-computed)); the data

generator, parameters, seeds and repeats are on page 1 and the count formulas on page 2.